

Advances in Spatial Hashing: A Pragmatic Approach towards Robust, Real-time Light Transport Simulation

Pascal Gautron
NVIDIA Corporation
USA
pgautron@nvidia.com

ABSTRACT

Spatial hashing is a battle-tested technique for efficiently storing sparse spatial data. Originally designed to optimize secondary light bounces in path tracing, it has been extended for real-time ambient occlusion and diffuse environment lighting. We complement spatial hashing by introducing support for view-dependent effects using world-space temporal filtering. Optimizing the hash key generation, we improve performance using a much better cache coherence and aliasing reduction. Finally, we enhance the sampling quality using methods including visibility-aware environment sampling.

CCS CONCEPTS

• Computing methodologies → Ray tracing.

ACM Reference Format:

Pascal Gautron. 2022. Advances in Spatial Hashing: A Pragmatic Approach towards Robust, Real-time Light Transport Simulation. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH '22 Talks)*, August 07-11, 2022. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532836.3536239>

Spatial hashing [Binder et al. 2022] is based on the computation of hash keys of discretized world-space attributes of a number of points in a scene. This algorithm provides near-constant fetch and storage times and is designed with parallelism in mind, making it the solution of choice for irradiance interpolation [Gautron 2021].

1 DUAL-SPACE FILTERING

Temporal filtering using image-space temporal reprojection is an attempt to match visible points from times $t - 1$ and t . We avoid the approximations and artifacts of temporal reprojection by extending spatial hashing to glossy surfaces. At time t each hash cell stores

$$L_t^H = \alpha \cdot L_{t-1}^H + (1 - \alpha) \cdot L_t, \quad (1)$$

the exponential average of the per-frame outgoing radiance L_t . During camera movement we use a small smoothing factor $\alpha \equiv \alpha_d$ to reduce ghosting. When navigation stops we gradually transition to a larger value α_s for temporal stability.

In motion the balance between temporal stability and ghosting highly depends on the surface materials. Smooth surfaces typically expose low noise but rapidly changing reflections. Hence, α_d should

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '22 Talks, August 07-11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9371-3/22/08.

<https://doi.org/10.1145/3532836.3536239>

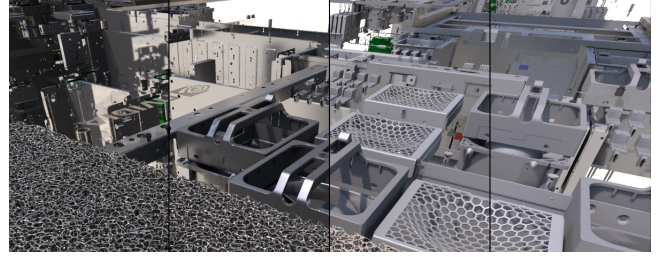


Figure 1: Servers scene (307M triangles) with varying roughness levels (0, 0.05, 0.2, 0.5), rendered at 90fps@1080p.

be small to avoid highlight ghosting. Conversely, sampling noise on rough surfaces is higher, but reflections change slowly. We model

$$\alpha_d(r) = r^q \quad (2)$$

to depend on the normalized surface roughness $r \in [0, 1]$, where q is a user-defined exponent (empirically $q = 0.2$ in our tests).

Our world-space temporal filtering now provides stability and denoising, but the remaining variance at low sample rates may create block artifacts due to the hashing discretization. These are resolved by an image-space filter inspired by the spatial component of Schied et al. [2017]. However, our approach mixes Boolean and scalar criteria, using hard limits on proximity and normal similarity while using smooth weights based on image-space distance and local variance resulting in a visually pleasing rendition.

2 OPTIMIZED HASH KEY GENERATION

2.1 Cache-Friendly Storage

The spatial hashing scheme applies an integer hash function H to the attributes of a world-space point, such as its coordinates p :

$$H_s(p) = H(\lfloor p_x/s \rfloor + H(\lfloor p_y/s \rfloor + H(\lfloor p_z/s \rfloor))), \quad (3)$$

where the integer s is the level of discretization. While achieving the required information spread, this also results in poor cache coherence. Grouping cells into world-space clusters mitigates this issue, at the expense of memory efficiency. Instead, we modify H_s to loosely preserve coherence over clusters of 2^3 voxels:

$$H_s^*(p) = H_{s-1}(p) + \left(\left\lfloor \frac{p_x}{s} \right\rfloor \% 2 \right) + 2 \left(\left\lfloor \frac{p_y}{s} \right\rfloor \% 2 \right) + 4 \left(\left\lfloor \frac{p_z}{s} \right\rfloor \% 2 \right). \quad (4)$$

Since $H_{s-1}(p)$ is not aligned on 8 entries, those clusters are not systematically allocated. Another unrelated world-space point may hash into that cluster, triggering collision mitigation. However, in general this approach optimizes memory usage and drastically improves cache performance.

2.2 Avoiding Aliasing

When an axis-aligned surface lies at the boundary between hash cells, floating-point arithmetic may associate neighboring points with distinct cells. Instead of hashing positions into cubic voxels we shift the coordinates of the points using a periodic function for each axis (Figure 2), reducing the likelihood of exactly matching the scene features. The corners of the cells must not be coplanar to avoid visible patterns: this is achieved using wavelengths that are not multiples of the original discretization step s :

$$H_s^{**}(p) = H_s^* \left(p + a \cdot \begin{pmatrix} F(p_x, s, 3) \\ F(p_y, s, 5) \\ F(p_z, s, 7) \end{pmatrix} \right), \quad (5)$$

where $F(x, s, n) = \sin \frac{2\pi x n}{s}$ and a is a small fraction of s .

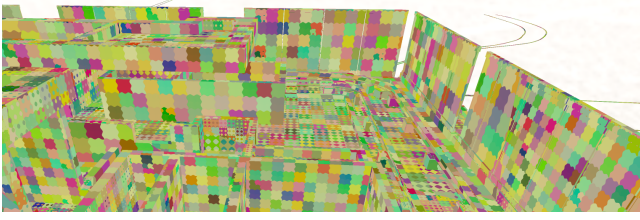


Figure 2: Large hash cells in false colors. The modified boundaries reduce aliasing and increase temporal stability.

3 IMPROVED SAMPLING

3.1 Visibility-Based Environment Sampling

Efficient importance sampling techniques for environment lighting [Binder and Keller 2019] provide fast convergence in outdoor scenes. However, in an interior scene most rays may be traced towards the sun despite occlusion by the ceiling. Here, ray guiding techniques improve efficiency at the cost of increased complexity.

We propose a simpler approach: the first N frames are rendered using classic importance sampling, while marking regions of the environment map that are visible to shadow rays. We then regenerate the importance sampling function from the marked regions only, and repeat this process every N frames. The cost of regenerating the importance function is affordable using the Cutpoint Method [Fishman and Moore 1984]: it takes 4.2ms for an 8MP environment map on a GeForce RTX 3090 and is further amortized by running this process every N frames (typically 10). We ensure domain exploration using multiple importance sampling of the environment map and the surface BSDF. In practice this coarse-grained optimization significantly increases the convergence speed (Figure 3).

3.2 Consistent Rendering

Each hash cell is associated to several pixels that are evaluated in separate threads, each of them tracing sample rays. Each such hash cell uses a single quasi-Monte Carlo sequence, with each thread reserving a sequence segment by incrementing an atomic counter [Gautron 2020]. Depending on thread scheduling, running the algorithm with the same inputs may generate a different image.

We propose to choose a stable, randomly chosen point within the area spanned by the hash cell, and trace all related rays from



Figure 3: Direct lighting of an elevator shaft in the Vancouver scene. Compared to the reference (middle), visibility-based sampling (right, overlay) provides a better reconstruction @10spp than importance sampling (left, overlay)

Table 1: Test scene statistics and rendering times with direct lighting, without and with optimized hash key generation.

Scene Name	Poly Count	Instance Count	Render (ms)		Speedup
			No Opt.	Opt.	
Vancouver	60M	220K	9.2	7.6	17.0%
Servers	307M	30K	11.9	9.9	16.8%

that point. To do so, we store two additional values per hash cell: a selector S and a pixel index P . The selector is a deterministic pseudorandom value obtained from the world-space coordinates p . Each thread computes its selector S and uses atomic functions to store the minimum S_{min} for that cell, along with its pixel index P_{min} . During raytracing we recover the world-space point p_{min} , and trace all rays from that point independent of thread scheduling.

In practice we pack S and P into a single 32-bit value, with the higher 7 bits storing S and the lower 25 bits storing P . This scheme supports up to 8K resolution, while the atomic minimum operation considers the selector and uses the pixel index in case of a draw.

4 RESULTS AND FUTURE WORK

Our implementation, based on the Vulkan API, renders complex assets (Table 1) at 1080p on an NVIDIA GeForce RTX 3090 (or any other RTX-class GPU), using a 140MB hashmap (5M cells).

Spatial hashing is a simple solution for ray traced lighting in massive scenes. Being world-space by nature, it avoids the pitfalls of temporal screen-space denoising. Work in progress includes extensions to global illumination and multi-GPU implementation.

REFERENCES

- Nikolaus Binder, Sascha Fricke, and Alexander Keller. 2022. Massively Parallel Path Space Filtering. In *Monte Carlo and Quasi-Monte Carlo Methods, MCQMC 2020, Oxford, United Kingdom, August 10–14*, Alexander Keller (Ed.). Springer.
- Nikolaus Binder and Alexander Keller. 2019. Massively Parallel Construction of Radix Tree Forests for the Efficient Sampling of Discrete Probability Distributions. *CoRR* abs/1901.05423 (2019). arXiv:1901.05423 <http://arxiv.org/abs/1901.05423>
- George S. Fishman and Louis R. Moore. 1984. Sampling from a Discrete Distribution while Preserving Monotonicity. *The American Statistician* 38, 3 (1984), 219–223.
- Pascal Gautron. 2020. Real-Time Ray-Traced Ambient Occlusion of Complex Scenes Using Spatial Hashing. In *ACM SIGGRAPH 2020 Talks*. Article 5. <https://doi.org/10.1145/3388767.3407375>
- Pascal Gautron. 2021. *Ray Tracing Gems II: Next Generation Real-Time Rendering with DXR, Vulkan, and OptiX*. Apress, Chapter Practical Spatial Hash Map Updates, 659–671. https://doi.org/10.1007/978-1-4842-7185-8_41
- Christoph Schied, Anton Kaplanyan, Chris Wyman, Anjul Patney, Chakravarty R. Alla Chaitanya, John Burgess, Shiqiu Liu, Carsten Dachsbacher, Aaron Lefohn, and Marco Salvi. 2017. Spatiotemporal Variance-Guided Filtering: Real-Time Reconstruction for Path-Traced Global Illumination. In *Proceedings of High Performance Graphics*. Article 2. <https://doi.org/10.1145/3105762.3105770>